Nome: Kevin Nogueira Meireles

Considere o vetor abaixo e o algoritmo **Shellsort** apresentado a seguir. Execute manualmente o algoritmo, registrando o vetor resultante após cada iteração do laço externo (linhas 3–13).

Índice	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**
Vetor inicial	76	17	83	26	59	11	74	35	68	20	91	47	13	56
Iteração 1	56	17	83	26	59	11	74	35	68	20	91	47	13	56
Iteração 2	13	11	74	26	56	17	83	35	59	20	91	47	68	76
Iteração 3	11	13	17	20	26	35	47	56	59	68	74	76	83	91
Iteração …														

Ao concluir, explique brevemente por que a última passagem do algoritmo garante a ordenação do vetor e qual é sua relação com o Insertion Sort.

```
SHELL-SORT(A, n)
1   h = 1
2   do h = h * 3 + 1 while (h < n)
3   do
4       h = h / 3
5       for i = h + 1 to n
6               x = A[i]
7               j = i
8               while A[j - h] > x do
9                       A[j] = A[j - h]
10                      j = j - h
```

```
11                  if j <= h break
12            A[j] = x
13  while (h != 1)
```

por que a última passagem garante a ordenação?

A última passagem utiliza intervalo $h = 1$, fazendo com que o algoritmo compare elementos vizinhos. Dessa forma, todos os elementos são inseridos em suas posições corretas, garantindo que o vetor fique completamente ordenado. Nessa etapa, o Shellsort funciona como o Insertion Sort, pois ambos utilizam o método de inserção para organizar os elementos.